



Christophe PICHAUD
Consultant sur les technologies Microsoft
christophepichaud@hotmail.com |
www.windowscpp.com



Space Invaders 1978 en C/C++ avec SFML

Ce mois-ci le gaming est à l'honneur. C'est un autre domaine dans lequel les hits sont écrits en C/C++. Nous allons nous intéresser à une bibliothèque qui se nomme SFML (Simple and Fast Multimedia Library). Il existe des bindings pour les autres langages comme Java, C#, Python, etc.



Le jeu d'arcade est un must. Qui n'a jamais tourné de l'œil devant un OutRun, Pacman, AfterBurner ou Space Invaders ? Parlons de ce dernier, sorti en 1978. Et si nous le recodions pour le fun sous Windows ?



Alors ? Prêt ?

SFML, la boîte à outils

Tout d'abord, il nous faut une bibliothèque qui supporte les objets graphiques animés ou *Sprites*, le chargement de textures ou images, etc. On pourrait faire ça avec les primitives Win32 du système d'exploitation mais SFML encapsule tout ça dans des classes C++ et c'est très bien fait.

SFML est une bibliothèque multimédia, ce qui veut dire quelle fournit une couche entre vous et le hardware. Elle est composée de 5 modules :

- System : c'est le module principal. Il propose les classes de vecteur 2D, 3D, les horloges, les threads, les strings, etc.
- Window : le module Window permet de créer des fenêtres, de

gérer l'interaction utilisateur, le clavier et la souris.

- Graphics : ce module fournit les fonctionnalités 2D comme le texte, les images, les formes et les couleurs.
- Audio : SFML propose un module pour la gestion du son.
- Network : ce module permet de créer des jeux en réseau et sur Internet et gère aussi HTTP ou FTP.

Commencer le codage du jeu

Avant tout il faut définir les fichiers d'entêtes nécessaires à utiliser. Nous les obtenons en téléchargeant SFML : <https://www.sfm-dev.org/download/sfml/2.5.1/>

Ensuite, créons une application console dans Visual Studio. Dans le fichier d'entêtes précompilées, ajoutons ça :

```
#include <stdio.h>
#include <tchar.h>
#include <string>
#include <sstream>
#include <memory>
#include <vector>

#include <SFML/Graphics.hpp>
#include <SFML/OpenGL.hpp>

#pragma comment(lib, "sfml-audio-d.lib")
#pragma comment(lib, "sfml-graphics-d.lib")
#pragma comment(lib, "sfml-main-d.lib")
#pragma comment(lib, "sfml-system-d.lib")
#pragma comment(lib, "sfml-window-d.lib")
#pragma comment(lib, "opengl32.lib")
```

On y inclut Graphics.hpp et c'est OK. On y inclut aussi OpenGL.hpp car le socle technique c'est OpenGL. SFML est multiplateforme. Les directives `pragma comment lib` indiquent au linker la liste des librairies qu'il faut lier à l'application. On y retrouve les

5 modules de SFML et OpenGL. Commençons par écrire le squelette de l'application : une routine main et une boucle principale.

```
int main()
{
    Game game;
    game.run();
}
```

Voyons le constructeur de la classe Game et les variables statiques :

```
const float Game::PlayerSpeed = 100.f;
const sf::Time Game::TimePerFrame = sf::seconds(1.f / 60.f);

Game::Game()
: m_Window(sf::VideoMode(840, 600), "Space Invaders 1978", sf::Style::Close)
, m_Texture()
, m_Player()
, m_Font()
, m_StatisticsText()
, m_StatisticsUpdateTime()
, m_StatisticsNumFrames(0)
, m_IsMovingUp(false)
, m_IsMovingDown(false)
, m_IsMovingRight(false)
, m_IsMovingLeft(false)
{
    m_Window.setFramerateLimit(160);

    BuildGameElements();
}
```

Le constructeur positionne la taille de la fenêtre principale.

La méthode BuildGameElements() construit le vaisseau du joueur, les blocs, les ennemis.

```
void Game::BuildGameElements()
{
    m_Texture.loadFromFile("Media/Textures/SI_Player.png");
    m_Player.setTexture(m_Texture);
    m_Player.setPosition(100.f, 500.f);

    std::shared_ptr<Entity> player = std::make_shared<Entity>();
    player->m_sprite = m_Player;
    player->m_type = EntityType::player;
    player->m_size = m_Texture.getSize();
    player->m_position = m_Player.getPosition();
    EntityManager::m_Entities.push_back(player);

    // Construction of enemies
    m_TextureEnemy.loadFromFile("Media/Textures/SI_Enemy.png");
    for (int i = 0; i < SPRITE_COUNT_X; i++)
    {
        for (int j = 0; j < SPRITE_COUNT_Y; j++)
        {
            m_Enemy[i][j].setTexture(m_TextureEnemy);
            m_Enemy[i][j].setPosition(
                100.f + 50.f * (i + 1),
```

```
0.f + 50.f * (j + 1));
```

```
        std::shared_ptr<Entity> se = std::make_shared<Entity>();
        se->m_sprite = m_Enemy[i][j];
        se->m_type = EntityType::enemy;
        se->m_size = m_TextureEnemy.getSize();
        se->m_position = m_Enemy[i][j].getPosition();
        EntityManager::m_Entities.push_back(se);
    }
}

// Construction of blocks
m_TextureBlock.loadFromFile("Media/Textures/SI_Block.png");
for (int i = 0; i < BLOCK_COUNT; i++)
{
    m_Block[i].setTexture(m_TextureBlock);
    m_Block[i].setPosition(0.f + 150.f * (i + 1), 350.f);

    std::shared_ptr<Entity> sb = std::make_shared<Entity>();
    sb->m_sprite = m_Block[i];
    sb->m_type = EntityType::block;
    sb->m_size = m_TextureBlock.getSize();
    sb->m_position = m_Block[i].getPosition();
    EntityManager::m_Entities.push_back(sb);
}

m_Font.loadFromFile("Media/Sansation.ttf");
m_StatisticsText.setFont(m_Font);
m_StatisticsText.setPosition(5.f, 5.f);
m_StatisticsText.setCharacterSize(10);
}
```

Chaque objet est une Entity et possède un type :

```
enum EntityType
{
    player,
    weapon,
    enemy,
    block
};
```

Voici la class Entity :

```
class Entity
{
public:
    Entity() {};
    ~Entity() {};

public:
    sf::Sprite m_sprite;
    sf::Vector2u m_size;
    sf::Vector2f m_position;
    EntityType m_type;
    bool m_enabled = true;
```

```
// Enemy only
bool m_bLeftToRight = true;
int m_times = 0;
};
```

Un objet graphique est un Sprite et possède une texture, une taille et une position. La classe Game encapsule le jeu et la méthode Run fait tout. Inspectons cette méthode Run() :

```
void Game::run()
{
    sf::Clock clock;
    sf::Time timeSinceLastUpdate = sf::Time::Zero;
    while (m_Window.isOpen())
    {
        sf::Time elapsedTime = clock.restart();
        timeSinceLastUpdate += elapsedTime;
        while (timeSinceLastUpdate > TimePerFrame)
        {
            timeSinceLastUpdate -= TimePerFrame;

            ProcessEvents();
            Update(TimePerFrame);
        }

        UpdateStatistics(elapsedTime);
        Render();
    }
}
```

En fait la routine principale est toujours la même. Pendant une durée déterminée, on gère les commandes utilisateurs et on anime les objets. De plus, la routine principale gère l'horloge du jeu. La méthode ProcessEvents gère les touches du clavier ; mouvement du joueur à droite, à gauche ou tir de missile :

```
void Game::ProcessEvents()
{
    sf::Event event;
    while (m_Window.pollEvent(event))
    {
        switch (event.type)
        {
            case sf::Event::KeyPressed:
                HandlePlayerInput(event.key.code, true);
                break;

            case sf::Event::KeyReleased:
                HandlePlayerInput(event.key.code, false);
                break;

            case sf::Event::Closed:
                m_Window.close();
                break;
        }
    }
}
```

Pour bien comprendre la gestion du tir, regardons la méthode HandlePlayerInput() qui est appelée sur la pression/relâche d'une touche :

```
void Game::HandlePlayerInput(sf::Keyboard::Key key, bool isPressed)
{
    if (key == sf::Keyboard::Up)
        m_IsMovingUp = isPressed;
    else if (key == sf::Keyboard::Down)
        m_IsMovingDown = isPressed;
    else if (key == sf::Keyboard::Left)
        m_IsMovingLeft = isPressed;
    else if (key == sf::Keyboard::Right)
        m_IsMovingRight = isPressed;

    // Creation of a weapon on SPACE
    if (key == sf::Keyboard::Space)
    {
        if (isPressed == false)
        {
            return;
        }

        std::shared_ptr<Entity> sw = std::make_shared<Entity>();
        sf::Texture texture;
        texture.loadFromFile("Media/Textures/SI_Weapon.png");
        sw->m_sprite.setTexture(texture);
        sw->m_sprite.setPosition(
            EntityManager::GetPlayer()->m_sprite.getPosition().x + EntityManager::
            GetPlayer()->m_sprite.getTexture()->getSize().x / 2,
            EntityManager::GetPlayer()->m_sprite.getPosition().y - 10);
        sw->m_type = EntityType::weapon;
        sw->m_size = sw->m_sprite.getTexture()->getSize();
        EntityManager::m_Entities.push_back(sw);
    }
}
```

Si on appuie sur Espace, on crée un objet weapon et on l'ajoute au gestionnaire d'objets EntityManager. EntityManager contient la liste des objets Entity. La méthode Update() gère la gestion des coordonnées et les propriétés des éléments par rapport à l'horloge.

```
void Game::Update(sf::Time elapsedTime)
{
    sf::Vector2f movement(0.f, 0.f);
    if (m_IsMovingUp)
        movement.y -= PlayerSpeed;
    if (m_IsMovingDown)
        movement.y += PlayerSpeed;
    if (m_IsMovingLeft)
        movement.x -= PlayerSpeed;
    if (m_IsMovingRight)
        movement.x += PlayerSpeed;

    // On each time event, move the PLAYER

    for (std::shared_ptr<Entity> entity : EntityManager::m_Entities)
```

```

{
    if (entity->m_enabled == false)
    {
        continue;
    }

    if (entity->m_type != EntityType::player)
    {
        continue;
    }

    entity->m_sprite.move(movement * elapsedTime.asSeconds());
}
}

```

La méthode Render() se charge d'afficher les éléments du jeu : Weapon, Player, Block, Enemy. Ils sont contenus dans EntityManager.

```

void Game::Render()
{
    m_Window.clear();

    for (std::shared_ptr<Entity> entity : EntityManager::m_Entities)
    {
        if (entity->m_enabled == false)
        {
            continue;
        }

        m_Window.draw(entity->m_sprite);
    }

    m_Window.draw(m_StatisticsText);
    m_Window.display();
}

```

Le cœur du jeu

La méthode UpdateStatistics() contient le moteur du jeu. Regardons le code et je vous explique :

```

void Game::UpdateStatistics(sf::Time elapsedTime)
{
    m_StatisticsUpdateTime += elapsedTime;
    m_StatisticsNumFrames += 1;

    if (m_StatisticsUpdateTime >= sf::seconds(1.0f))
    {
        m_StatisticsText.setString(
            "Frames / Second = " + toString(m_StatisticsNumFrames) + "\n" +
            "Time / Update = " + toString(m_StatisticsUpdateTime.asMicroseconds() / m_StatisticsNumFrames) + "us");

        m_StatisticsUpdateTime -= sf::seconds(1.0f);
        m_StatisticsNumFrames = 0;
    }
}

```

```

//
// Handle collision
//

if (m_StatisticsUpdateTime >= sf::seconds(0.050f))
{
    for (std::shared_ptr<Entity> weapon : EntityManager::m_Entities)
    {
        if (weapon->m_enabled == false)
        {
            continue;
        }

        if (weapon->m_type != EntityType::weapon)
        {
            continue;
        }

        for (std::shared_ptr<Entity> enemy : EntityManager::m_Entities)
        {
            if (enemy->m_type != EntityType::enemy)
            {
                continue;
            }

            if (enemy->m_enabled == false)
            {
                continue;
            }

            // Check for collision between weapon and enemies

            sf::FloatRect boundWeapon;
            boundWeapon = weapon->m_sprite.getGlobalBounds();

            sf::FloatRect boundEnemy;
            boundEnemy = enemy->m_sprite.getGlobalBounds();

            if (boundWeapon.intersects(boundEnemy) == true)
            {
                enemy->m_enabled = false;
                weapon->m_enabled = false;
                //break;
                goto end;
            }
        }
    }

end:

//
// Handle Weapon moves
//

```

```

for (std::shared_ptr<Entity> entity : EntityManager::m_Entities)
{
    if (entity->m_enabled == false)
    {
        continue;
    }

    if (entity->m_type != EntityType::weapon)
    {
        continue;
    }

    float x, y;
    x = entity->m_sprite.getPosition().x;
    y = entity->m_sprite.getPosition().y;
    y--;

    if (y <= 0)
    {
        entity->m_enabled = false;
    }
    else
    {
        entity->m_sprite.setPosition(x, y);
    }
}

//
// Handle Enemy moves
//

for (std::shared_ptr<Entity> entity : EntityManager::m_Entities)
{
    if (entity->m_enabled == false)
    {
        continue;
    }

    if (entity->m_type != EntityType::enemy)
    {
        continue;
    }

    float x, y;
    x = entity->m_sprite.getPosition().x;
    y = entity->m_sprite.getPosition().y;

    if (entity->m_bLeftToRight == true)
        x++;
    else
        x--;
    entity->m_times++;

    if (entity->m_times >= 50)
    {

```

```

        if (entity->m_bLeftToRight == true)
        {
            entity->m_bLeftToRight = false;
            entity->m_times = 0;
        }
        else
        {
            entity->m_bLeftToRight = true;
            entity->m_times = 0;
        }
    }

    entity->m_sprite.setPosition(x, y);
}
}

```

Cette méthode contient 3 parties :

- La gestion des collisions ;
- La gestion des tirs de missiles (weapons) ;
- La gestion du mouvement des ennemis.

Pour gérer la collision entre un weapon et les ennemis, on prend les rectangles de positions disponibles via `Sprite::getGlobalBounds` sur weapon et enemy et on utilise la méthode `FloatRect::intersects`. Si elle retourne true, il y a collision et dans notre cas, on positionne le flag `m_enemy` des deux éléments, ce qui fait qu'ils ne seront plus affichés. Ce 3 parties sont des routines qui itèrent sur la collection des objets (`EntityManager`), et, suivant le type des objets, performent des opérations ou modifient des états comme `Enabled` pour désactiver l'entity.

Le résultat

Voici à quoi ça ressemble :



Conclusion

Le code du jeu est disponible sur GitHub dans <https://github.com/ChristophePichaud/SpaceInvaders>. Le code n'est pas très compliqué une fois que la boucle principale est comprise. J'espère que cet article vous donne envie de tester SFML en C++ ou en C#.